

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>
int main() {
    char cmd[100];
    pid_t pid;
    printf("Enter a Linux command: ");
    scanf("%s", cmd);
    pid = fork();
    if (pid < 0) {
        printf("Fork failed!\n");
        return 1;
    }
    else if (pid == 0) {
        printf("\nChild Process\n");
        printf("Child PID : %d\n", getpid());
        printf("Parent PID : %d\n", getppid());
        char *args[] = {cmd, NULL};
        execvp(cmd, args);
        exit(1);
    }
    else {
        printf("\nParent Process\n");
        printf("Parent PID : %d\n", getpid());
        printf("Child PID : %d\n", pid);
        wait(NULL);
        printf("Child process completed.\n");
    }
    return 0;
}
```

output:

Enter a Linux command: ls

Parent Process

Parent PID : 98520

Child PID : 98525

Child Process

Child PID : 98525

Parent PID : 98520

```
addition      arithmetic.c  execworking.c forkid      linuxos.c      multiprocess2
multiprocess3.c  process2    process3.c  process5    switchcase.c  zombie
addition.c      dir3        f1.txt      forkid.cmultiprocess  multiprocess2.c
process         process2.c  process4    process5.c  write         zombie.c
arithmetic      execworking  f4.txt      linuxos     multiprocess.c multiprocess3
process.c       process3    process4.c  switchcase  write.c
Child process completed.
```